

ABSTRACT

The rapid growth of digital documents in educational, professional, and organizational environments has created a need for intelligent systems that can retrieve relevant information quickly and accurately. Traditional keyword-based search often fails to provide direct answers, especially when users need concise responses from lengthy documents. To address this issue, this mini project presents **MindVault**, an intelligent document assistant that allows users to upload documents and ask natural language questions based on their content.

The proposed system is based on the concept of **Retrieval-Augmented Generation (RAG)**, in which the application first retrieves the most relevant chunks of text from uploaded documents and then generates an answer grounded in that retrieved context. The system supports multiple document formats such as **PDF, DOCX, PPT, and PPTX**, thereby making it flexible for a variety of academic and professional use cases. It also provides secure **user authentication, session-based conversation history, and document-specific chat selection**, enabling multiple users to manage their own documents and conversations independently.

The backend of the system is implemented using **FastAPI, SQLAlchemy, and SQLite** for API services and data persistence. **ChromaDB** is used as the vector database for storing embeddings, while **SentenceTransformers** is used for generating semantic embeddings in local mode. The frontend is built using **React, Vite, and Tailwind CSS** to provide a responsive and user-friendly interface.

The developed system demonstrates how modern AI-powered retrieval techniques can improve human-computer interaction for document understanding tasks. MindVault reduces the effort needed to manually search through long files and provides context-aware responses with source information. The project can be extended further by adding OCR support, multilingual processing, voice interaction, and deployment in cloud environments.

TABLE OF CONTENT

Chapter No.	Title	Page No.
1	Introduction	1
1.1	Problem Statement	1
1.2	Project Summary and Introduction	1–2
1.3	Aim and Objectives	2
2	System Analysis	3
2.1	Motivation	3
2.2	Literature Survey	3–4
3	Design: Analysis & Methodology	5
3.1	Hardware & Software Requirements	5
3.2	Module Specification	6
3.3	Methodology	7–8
3.4	Timeline Chart	9
3.5	UML Diagrams	10
4	Implementation	14
4.1	System Flow	14
4.2	Database Design	14–15
4.3	Module Specification with Screenshots	16–20
4.4	Results	21
5	Conclusion & Future Work	23
-	References	24

LIST OF FIGUERS

Figure No.	Title	Page No.
Fig 3.1	Use Case Diagram	10
Fig 3.2	Activity Diagram	11
Fig 3.3	Class Diagram	12
Fig 3.4	Sequence Diagram	13
Fig 4.1	ER Diagram	15

LIST OF TABLES

Table No.	Title	Page No.
Table 3.1	Hardware and Software Requirements	5
Table 3.2	Module Specification	6
Table 3.3	Timeline Chart	9
Table 4.1	Database Tables Description	14
Table 4.2	Functional Results Summary	21

CHAPTER 1

INTRODUCTION

1.1 Problem Statement

In the present digital era, a large amount of information is stored in the form of electronic documents such as PDF files, Word documents, and presentation slides. Although this information is readily available, extracting meaningful and specific information from large documents remains difficult for users. Conventional searching methods depend mainly on keyword matching, and therefore they often fail to provide precise answers when the user asks a question in natural language. This leads to time consumption, reduced productivity, and difficulty in information access, especially in academic, technical, and administrative domains.

Another major issue is that many existing chatbot systems are general-purpose and generate answers without directly grounding them in user-provided documents. As a result, they may produce irrelevant or misleading responses. In practical scenarios, users require a system that can answer questions specifically from the documents they upload, while also maintaining privacy, user-specific access control, and conversational continuity.

The problem addressed in this project is the development of a secure and intelligent document assistant that can accept user-uploaded files, process their content, retrieve relevant information semantically, and generate accurate answers based on that information. The system should support multiple document formats, preserve individual chat sessions, and provide a simple interface for interaction.

1.2 Project Summary and Introduction

The proposed mini project, **MindVault**, is an intelligent document-based chatbot that combines document retrieval and conversational AI. The system enables users to register, log in securely, upload documents, create chat sessions, and ask questions related to uploaded files. The backend extracts textual content from the uploaded documents, converts the content into meaningful semantic embeddings, stores them in a vector database, and retrieves the most relevant pieces of information when a query is asked. The retrieved context is then passed to a response generation model to produce a final answer.

This project follows the principle of **Retrieval-Augmented Generation**, which is widely used in modern AI systems for context-grounded answering. Instead of relying only on the language model's internal knowledge, the system first searches through indexed document chunks and then generates the answer from those retrieved chunks. This improves answer reliability and makes the system useful for document understanding tasks.

MindVault also includes additional useful features such as user authentication through JSON Web Tokens, session management for storing multiple

conversations, automatic session title generation, document-wise context selection, and source chunk display in responses. The frontend provides a responsive

environment where users can browse sessions, upload files, and chat with the assistant.

The project has practical value in academic institutions, offices, legal environments, and enterprise knowledge systems, where users frequently need quick answers from lengthy documents. It reduces manual effort, improves information access, and demonstrates how AI can be integrated into document-centric workflows.

1.3 Aim and Objective of the Project

The main aim of this project is to build a secure and efficient AI-powered document assistant that allows users to interact with their uploaded documents using natural language.

The objectives of the project are as follows:

1. To develop a web-based system for uploading and managing documents.
2. To support multiple document formats such as PDF, DOCX, PPT, and PPTX.
3. To implement secure user registration and login functionality.
4. To extract text from uploaded documents and convert it into searchable vector representations.
5. To apply retrieval-augmented generation for accurate and context-based answering.
6. To maintain session-wise chat history for better conversation continuity.
7. To provide a simple and user-friendly interface for document interaction.
8. To display document-based responses along with source references.
9. To build the system using open and modern technologies suitable for future extension.
10. To demonstrate the usefulness of AI in intelligent information retrieval and question answering.

CHAPTER 2

SYSTEM ANALYSIS

2.1 Motivation

The motivation for this project comes from the increasing need to interact intelligently with stored documents. Students, researchers, teachers, and professionals often work with large collections of notes, reports, manuals, presentations, and reference material. Searching these documents manually is inefficient and often frustrating, especially when the required information is hidden inside lengthy pages or spread across multiple sections.

Most users prefer asking direct questions such as “What is the conclusion of this report?” or “What are the key points in chapter 3?” rather than opening the file and reading it line by line. Existing search mechanisms can highlight matching words, but they usually do not understand the semantic meaning of the query. This creates a gap between human language and machine search.

The emergence of large language models and vector retrieval systems has made it possible to build document assistants that can understand user questions and answer from the actual content of uploaded files. This project is motivated by the desire to apply these modern techniques in a practical mini project that is useful, technically relevant, and aligned with current industry trends.

The project is also motivated by the need for personalization and privacy. Instead of using a public chatbot detached from user data, MindVault organizes uploads per user and restricts access using authentication and session ownership. This makes the application more realistic and suitable as a prototype for secure knowledge systems.

2.2 Brief Literature Survey

Document question answering and intelligent retrieval have evolved significantly over the years. Early systems depended on rule-based approaches and keyword matching, which were limited in flexibility and often unable to capture the real meaning of a query. These methods were efficient for exact term matching but performed poorly when the wording of the question differed from the wording in the document.

With the development of machine learning and deep learning, semantic understanding became possible through embeddings and transformer-based language models. Models such as BERT and SentenceTransformers enabled text to be represented as dense vectors, where semantically similar content appears close in vector space. This allowed more accurate retrieval compared to classical keyword systems.

A major advancement came with the concept of **Retrieval-Augmented Generation**, in which a language model is supplied with externally retrieved context before generating the response. This architecture helps reduce hallucination

and improves factual relevance. Research by Patrick Lewis and others on RAG demonstrated that combining retrieval with generation provides better grounded answers in knowledge-intensive tasks.

Vector databases such as ChromaDB, FAISS, and Pinecone further enabled efficient storage and retrieval of embeddings at scale. These tools made it practical to build systems where uploaded content can be split into chunks, embedded, indexed, and searched semantically in real time.

Recent web development frameworks such as FastAPI and React have also simplified the development of AI-backed applications. FastAPI provides high-performance backend APIs with data validation, while React offers interactive user interfaces suitable for chat-based applications.

Based on this literature and technology trend, the present project adopts a retrieval-augmented architecture using text chunking, embeddings, vector storage, and response generation to build a document assistant. The project combines the theoretical ideas of semantic retrieval and generative AI with a practical full-stack implementation.

CHAPTER 3

DESIGN: ANALYSIS, DESIGN METHODOLOGY

3.1 Hardware and Software Requirement

The development and execution of MindVault require moderate hardware and commonly used software tools. The system is designed so that it can run on a typical student laptop or desktop computer.

Table 3.1 Hardware and Software Requirements

Category	Requirement
Processor	Intel Core i3/i5 or equivalent
RAM	Minimum 8 GB
Storage	Minimum 10 GB free space
Operating System	Windows 10/11 or equivalent Linux environment
Backend Language	Python 3.10 or above
Frontend Language	JavaScript
Backend Framework	FastAPI
ORM	SQLAlchemy
Database	SQLite
Vector Store	ChromaDB
Embedding Model	SentenceTransformers
Document Parsing Libraries	PyPDF, python-docx, python-pptx
Authentication	JWT, bcrypt
Frontend Framework	React
Build Tool	Vite
Styling	Tailwind CSS

Category	Requirement
HTTP Client	Axios
Development IDE	Visual Studio Code
Browser	Google Chrome / Microsoft Edge

The backend is responsible for authentication, file processing, retrieval, and API management. The frontend handles user interaction, file upload, session view, and chat rendering. SQLite is used as a lightweight relational database for storing users, sessions, and messages. ChromaDB is used as the persistent vector store for document embeddings.

3.2 Program or Module Specification

The system is divided into modular components for better maintainability and separation of concerns.

Table 3.2 Module Specification

Module	Description
User Authentication Module	Handles user registration, login, password hashing, token generation, and identity verification
Document Upload Module	Accepts document files from the user and validates supported file formats
Text Extraction Module	Extracts textual content from PDF, DOCX, PPT, and PPTX files
Chunking Module	Splits extracted text into manageable overlapping chunks
Embedding Module	Converts chunks and queries into vector embeddings
Vector Storage Module	Stores chunk embeddings along with metadata such as source file and page number
Retrieval Module	Fetches the most relevant chunks based on semantic similarity
Answer Generation	Generates grounded answers using retrieved

Module	Description
Module	context
Session Management Module	Creates, renames, lists, and deletes chat sessions
Message Management Module	Stores user and assistant messages for each session
Frontend UI Module	Provides screens for login, upload, session list, and chat interaction

The overall architecture follows a client-server approach. The frontend communicates with the backend through HTTP APIs. The backend further interacts with both a relational database and a vector database.

3.3 Methodology

The methodology adopted for this project is based on the Retrieval-Augmented Generation workflow. The complete operation can be described in several stages.

First, the user creates an account or logs in to the system. After successful authentication, the user uploads one of the supported document formats. The backend validates the file and stores it in a user-specific directory. Then the document is parsed according to its type. PDF files are processed page by page, Word documents are read paragraph-wise and grouped into sections, and presentation files are processed slide by slide.

Once the text is extracted, it is split into chunks of manageable size. Chunking is important because large documents cannot be processed efficiently as a single block. Overlapping chunks are created so that context continuity is preserved across boundaries. Each chunk is then converted into a vector representation using an embedding model. These vectors are stored in ChromaDB along with metadata such as file name, page number, chunk index, and user identifier.

When the user asks a question, the system first converts the query into an embedding and performs similarity search in the vector database. The top relevant chunks are retrieved based on semantic closeness. These chunks are then assembled as contextual information for answer generation. The response is produced either using a local model, an OpenAI-compatible model, or an Ollama-served model depending on configuration. The final answer is returned to the user along with source chunk references.

The system also maintains chat history and associates messages with sessions. This allows users to continue previous conversations and keep their interactions organized. Security is ensured by token-based authentication and user-specific data filtering.

The adopted methodology is suitable because it improves answer relevance, supports multi-document interaction, and separates storage, retrieval, and generation responsibilities in a clear manner.

Algorithmic Steps

1. User registers or logs in.
2. User uploads a document.
3. System validates document extension.
4. Text is extracted according to file type.
5. Extracted text is divided into overlapping chunks.
6. Each chunk is converted into an embedding vector.
7. Vectors and metadata are stored in ChromaDB.
8. User creates or selects a chat session.
9. User asks a natural language question.
10. System converts the query into an embedding.
11. Relevant chunks are retrieved from the vector store.
12. Retrieved chunks are passed as context to the answer generation component.
13. Final answer and sources are displayed to the user.
14. Messages are stored in the session for future reference.

3.4 Timeline Chart

The project was completed in planned phases to ensure systematic progress from requirement analysis to final testing.

Table 3.3 Timeline Chart

Phase	Activity	Duration
Phase 1	Studied Project domain & similar system . Finalized problem and selected project	Week 1
Phase 2	Worked on objectives, user roles & system workflow and created a basic architecture.	Week 2
Phase 3	Explored key features like semantic retrieval, multi pdf knowledge and document parsing.	Week 3
Phase 4	Worked on Functional & non-Functional Requirement.	Week 4
Phase 5	Designed basic UI using Streamlite.	Week 5
Phase 6	Modified UI & added PDF upload & Storing feature.	Week 6
Phase 7	Worked on text processing.	Week 7
Phase 8	Worked on text chunks conversion into vector embeddings using LLM & LLM integration.	Week 8
Phase 9	Worked on generated embeddings storing in vector database using FAISS.	Week 9
Phase 10	Implemented retrieval mechanisms for user questions & searches the vector database to find relevant text.	Week 10
Phase 11	Worked on LLM implementation.	Week 11
Phase 12	Added ppt, pptx and docs feature.	Week 12
Phase 13	Changed UI completely made login, signup and new dashboard look.	Week 13
Phase 14	Testing and Documentation.	Week 14

3.5 UML Diagrams

Use Case Diagram

The major actors and use cases of the system are as follows:

Primary Actor: Registered User

Use Cases:

1. Register account
2. Log in
3. Upload document
4. View uploaded documents
5. Create new chat session
6. Select an existing session
7. Ask question
8. Receive answer with sources
9. Rename session
10. Delete session
11. Log out

Use Case Diagram for Document Chat System

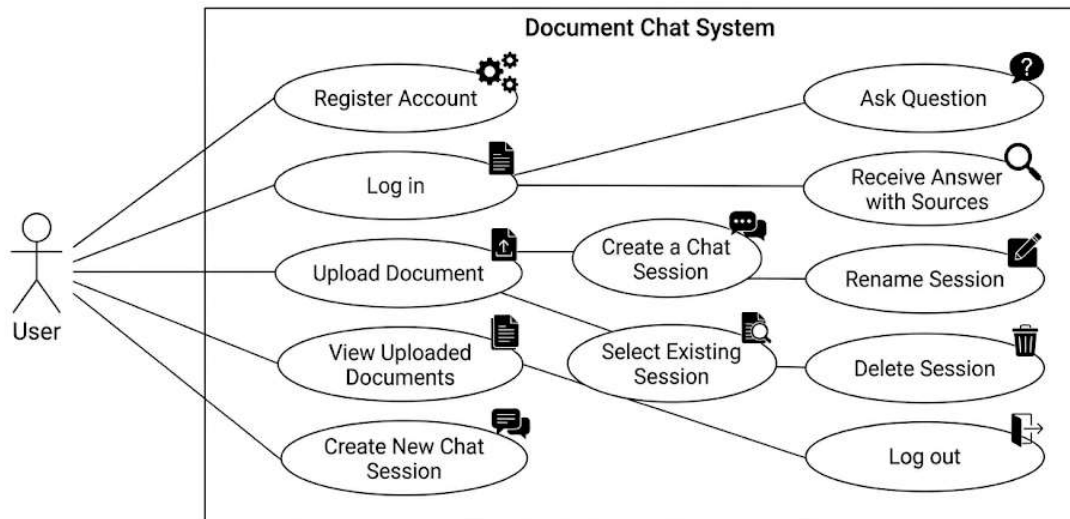


Fig.3.1 Use case

Use Case Description

The registered user interacts with the application through the web interface. The user authenticates using email and password, uploads a document, and starts a new chat session. The system processes the uploaded file, indexes the content, and answers user queries by retrieving relevant document chunks. The user can manage multiple sessions and revisit earlier conversations.

Activity Diagram

The activity flow of the system is as follows:

1. Start
2. User opens application
3. User registers or logs in
4. User uploads document
5. System validates and processes document
6. User selects document context
7. User enters question
8. System retrieves relevant chunks
9. System generates contextual answer
10. System displays answer and sources
11. User continues chat or exits
12. End

Mindvault - A pdf Chatbot

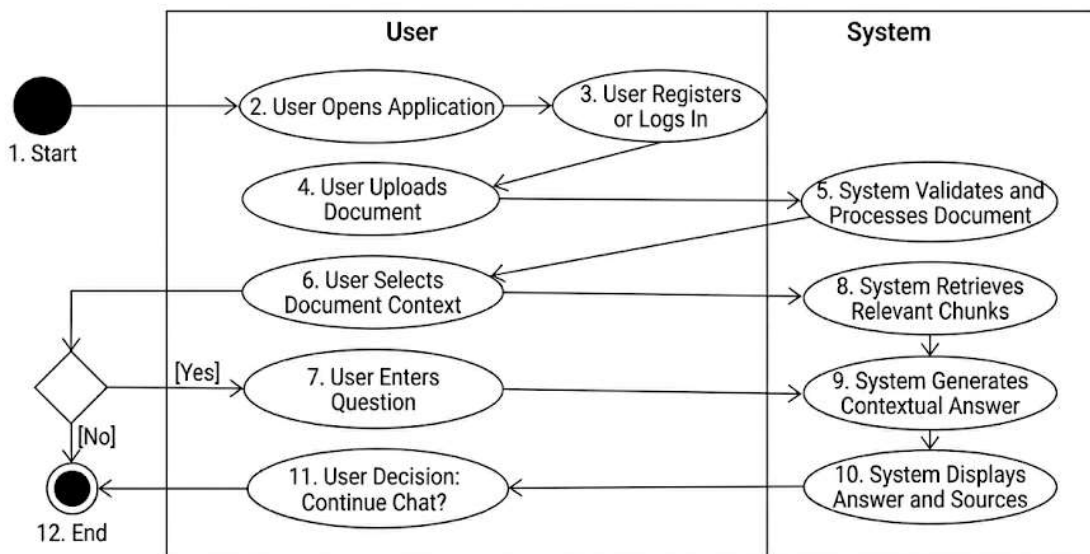


Fig.3.2 Activity Diagram

Class Diagram

The major classes or entities in the system are:

1. User
2. ChatSession
3. Message
4. RAGService
5. Settings
6. API service components

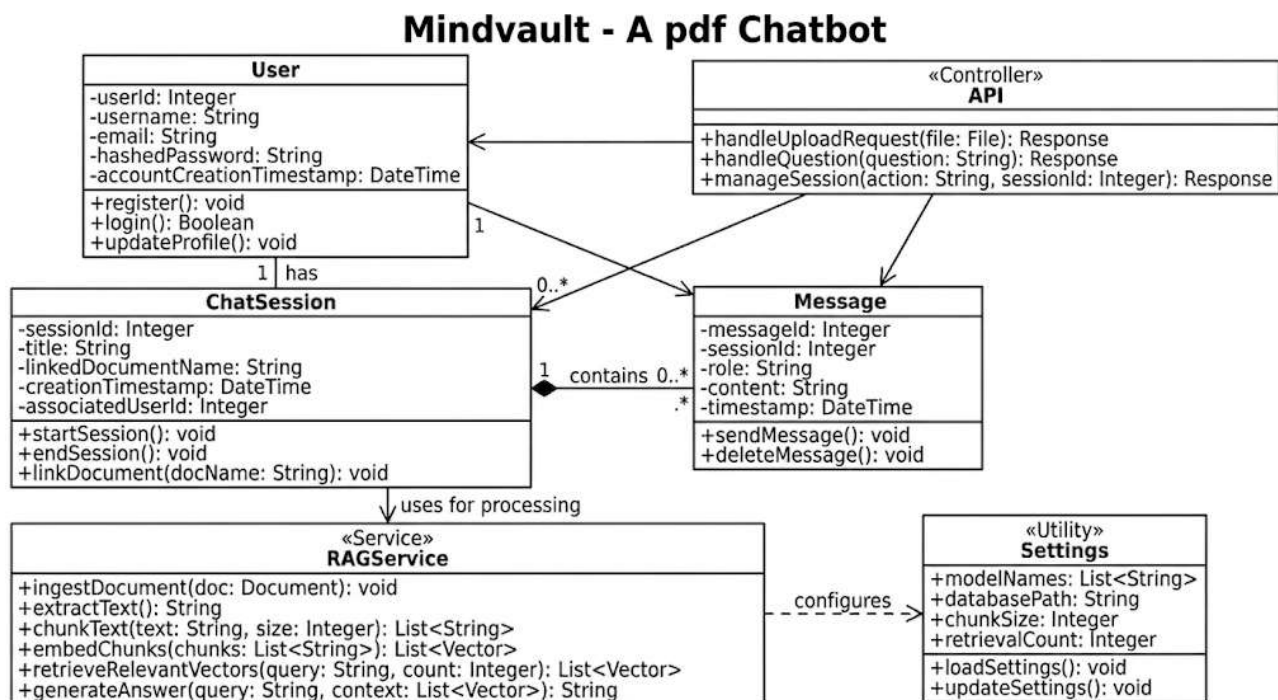


Fig.3.3 Class Diagram

User contains fields such as user ID, username, email, hashed password, and account creation timestamp.

ChatSession contains session ID, title, linked document name, creation timestamp, and the associated user.

Message contains message ID, session ID, role, content, and timestamp.

RAGService performs document ingestion, extraction, chunking, embedding, retrieval, and answer generation.

Settings stores configuration values such as model names, database path, chunk size, and retrieval count.

Sequence Diagram

The sequence of interactions can be explained as follows:

1. User sends upload request from frontend.
2. Backend validates file and extracts text.
3. Chunking and embedding are performed.
4. Chunks are stored in vector database.
5. User sends a question.
6. Backend retrieves relevant vectors.
7. Context is passed to language model.
8. Generated answer is returned to frontend.
9. Frontend displays answer and source details.

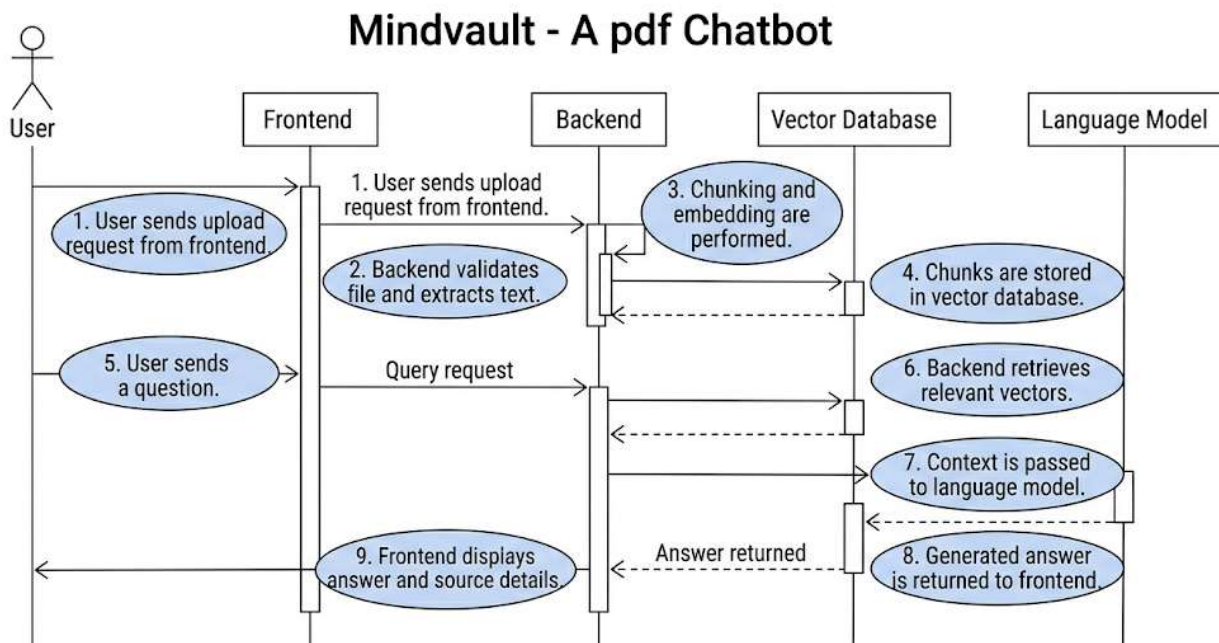


Fig.3.4 Class Diagram

CHAPTER 4

IMPLEMENTATION

4.1 System Flow

The implementation of MindVault follows a structured flow from user authentication to answer generation. The application begins with the login and registration page. When a new user registers, the password is hashed before storage, and on successful login a token is generated for future authorized requests. This ensures that all subsequent operations are performed in a secure, user-specific context.

After logging in, the user enters the home screen that contains the chat interface, session sidebar, and upload option. The user may upload a new document in supported formats. The backend stores the file in a user-specific folder and extracts text from the file. Different extraction strategies are used based on file type so that the architecture remains flexible and modular.

The extracted text is then chunked into overlapping pieces. The reason for using chunking is to improve retrieval quality and manage model context efficiently. Each chunk is embedded into vector space and stored persistently in ChromaDB with metadata. If the same user uploads a file with the same name again, the system can replace the old indexed chunks so that outdated and duplicate embeddings do not remain.

When the user starts a new chat or opens an existing one, a session record is maintained in the relational database. User messages and assistant responses are stored as part of session history. If the conversation title is still the default value, the system automatically generates a new title from the first question asked by the user.

When a question is asked, the backend first confirms that the requested session belongs to the logged-in user. It then embeds the query, searches the vector database, retrieves the top relevant chunks, and builds a context. That context is passed to the answer generation layer. The answer is sent back to the frontend along with source details such as document name and page number, allowing the user to understand the origin of the response.

The frontend updates the message view, scrolls automatically to the latest answer, and refreshes the session list so that recent conversations appear first. This creates a smooth user experience and maintains conversation continuity.

4.2 Database Design

The system uses a relational database for transactional data and a vector database for semantic retrieval.

Relational Database

SQLite is used to store structured data. The following tables are maintained.

Table 4.1 Database Tables and Description

Table Name	Purpose	Important Fields
Users	Stores registered user information	id, username, email, hashed_password, created_at
Chat_Sessions	Stores conversation sessions of each user	id, user_id, title, document_name, created_at
Messages	Stores messages for each session	id, session_id, role, content, timestamp

Entity Relationship Description

A single user can have many chat sessions.

A single chat session can have many messages.

Each message belongs to exactly one chat session.

This defines the following relationships:

1. User to ChatSession is one-to-many.
2. ChatSession to Message is one-to-many.

Mindvault - A pdf Chatbot Database Schema

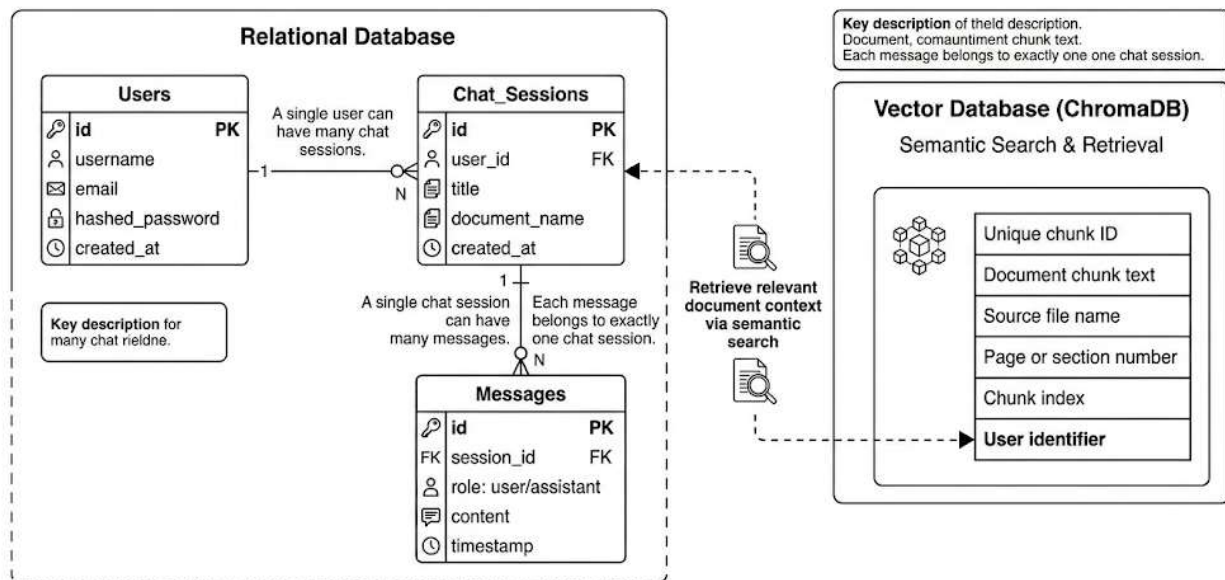


Fig 4.1 ER-Diagram

Vector Database

ChromaDB is used as the vector store for document retrieval. Each stored vector entry contains:

1. Unique chunk ID
2. Document chunk text
3. Source file name
4. Page or section number
5. Chunk index
6. User identifier

This separation of relational and vector storage is useful because structured application data and semantic search data have different access patterns.

Data Dictionary

Users Table

1. id: Unique identifier for each user
2. username: User's display name
3. email: User login email
4. hashed_password: Encrypted password
5. created_at: Account creation time

Chat_Sessions Table

1. id: Unique session identifier
2. user_id: Reference to owner user
3. title: Session title
4. document_name: Currently associated document
5. created_at: Session creation time

Messages Table

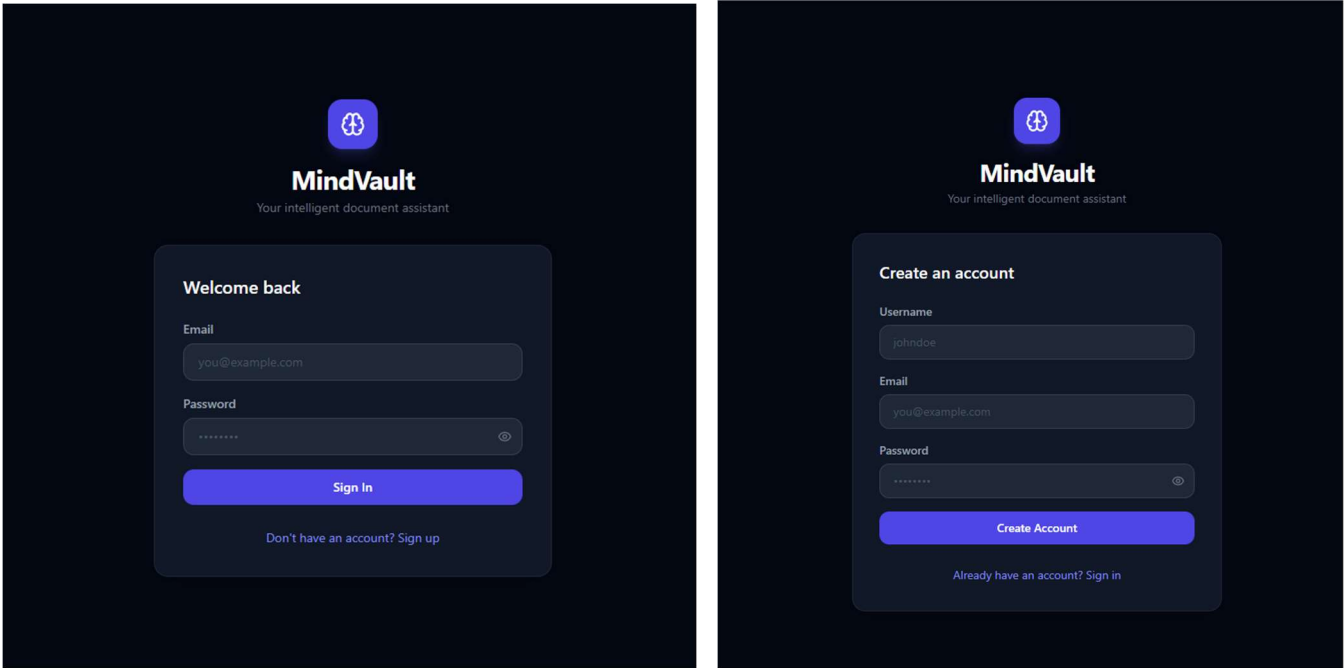
1. id: Unique message identifier
2. session_id: Reference to parent session
3. role: Indicates whether message is from user or assistant
4. content: Message text
5. timestamp: Time of message creation

4.3 Module Specification with Screenshot of Working Module

1. Authentication Module

This module provides user registration and login features. Passwords are not stored in plain text. Instead, hashing is used to improve security. After successful login, the user receives a token that is attached to future API requests. This module ensures that user-specific documents and sessions remain protected.

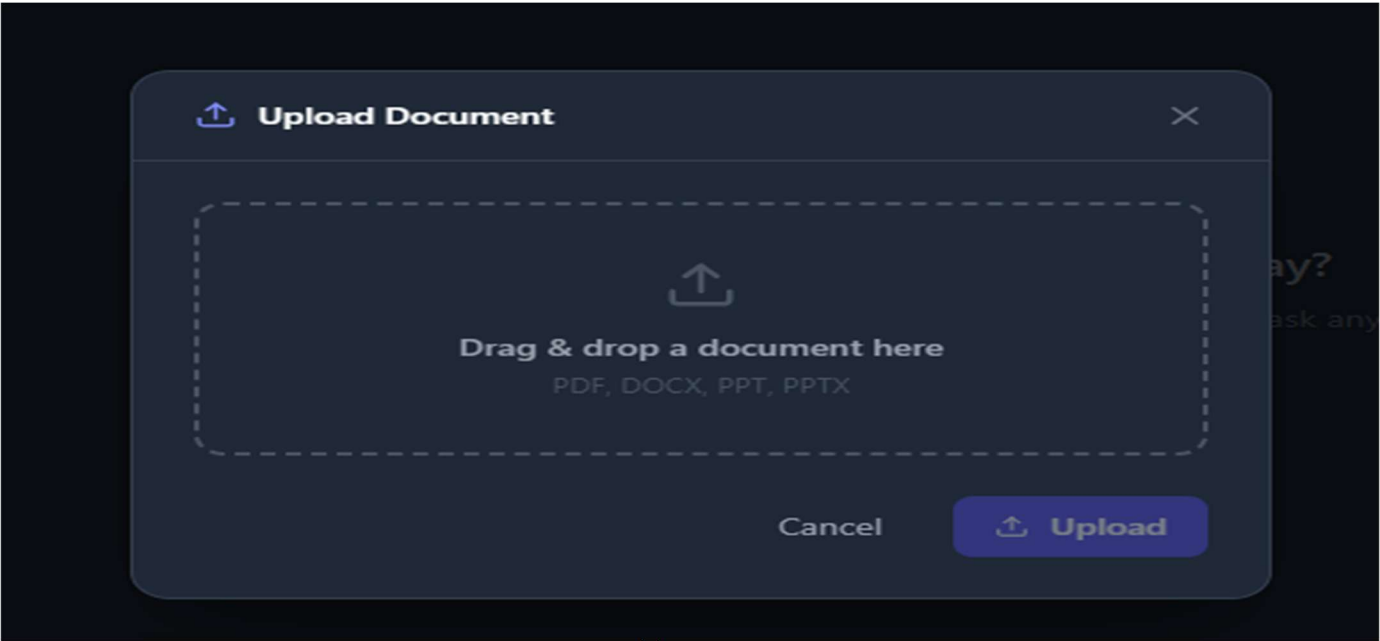
Screenshot: Login and registration page



2. Upload Module

This module allows the user to upload supported document files. The frontend provides a drag-and-drop or file selection interface. The backend validates the file extension and processes only supported formats. After a successful upload, the system displays confirmation and the number of chunks indexed.

Screenshot: Upload document modal



3. Document Processing Module

This module reads the content of the uploaded file. PDFs are processed page-wise, Word files are processed by paragraph groups, and presentation files are processed slide-wise. The extracted text is cleaned and prepared for chunking. This module acts as the bridge between raw documents and semantic retrieval.

4. Retrieval Module

This is the core intelligent component of the system. It transforms text chunks into embeddings and stores them in ChromaDB. When a user asks a question, the query is also embedded, and the most relevant chunks are retrieved through similarity search. This module ensures that the answer is based on actual uploaded content instead of generic model memory.

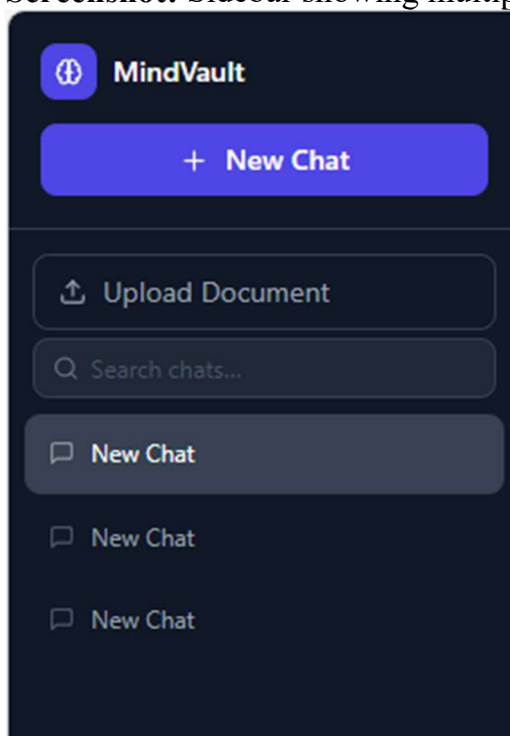
5. Response Generation Module

After retrieval, the relevant chunks are combined into a context prompt. A configured language model then generates a response from this context. If there is no matching document content, the system returns a clear fallback response. In summary-style queries, the system can gather more chunks to produce broader answers.

6. Session Management Module

The session management module allows the user to create new chats, rename existing sessions, delete sessions, and revisit old conversations. This improves usability and helps organize multiple document discussions.

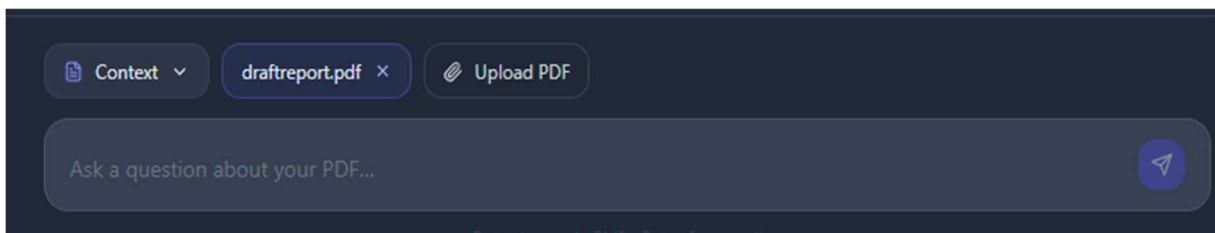
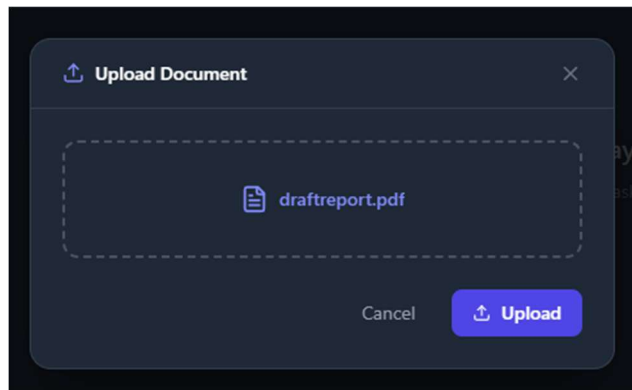
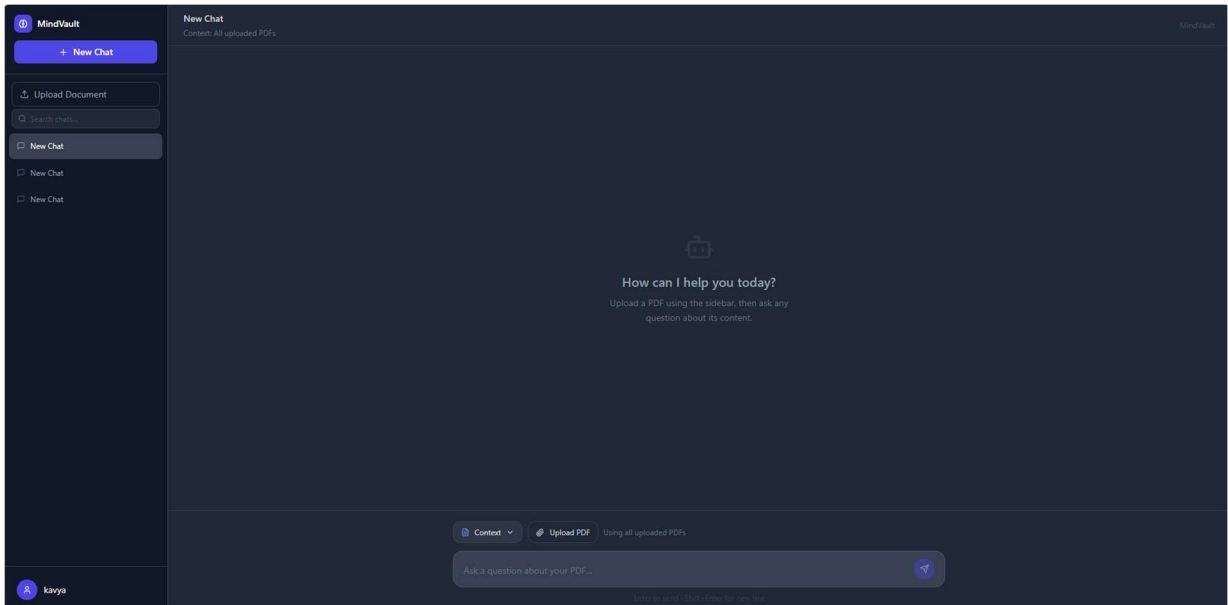
Screenshot: Sidebar showing multiple chat session



7. Chat Interface Module

This module displays messages between the user and the assistant. It includes optimistic rendering of user messages, loading indication while waiting for the response, and the final assistant answer with source references. The design resembles a modern conversational interface and improves user engagement.

Screenshot: Chat window with user question and document-based answer





Based on the provided partial summaries, here is a clean final answer with the requested sections:

The MindVault project is an intelligent document assistant that retrieves relevant information from uploaded documents using Retrieval-Augmented Generation (RAG) technology. The system aims to provide concise answers grounded in the retrieved context, reducing manual search effort and improving information access.

- * The MindVault system combines document retrieval and conversational AI to enable users to upload documents, create chat sessions, and ask questions related to uploaded files.
- * The system uses Retrieval-Augmented Generation to improve answer reliability and make it useful for document understanding tasks.
- * Key features include user authentication through JSON Web Tokens, session management for storing multiple conversations, automatic session title generation, document-wise context selection, and source chunk display in responses.

- * The project was carried out under guidance from Prof. XYZ as part of the Bachelor of Engineering in Computer Engineering degree at G H Patel College of Engineering & Technology, CVM University.
- * The report is certified by Dr. Zubin Bhaidasna and Dr. Sudhir Vegad, the internal guide and head of department respectively.
- * The system's technical implementation includes a backend built using FastAPI, SQLAlchemy, and SQLite for API services and data persistence, while the frontend is built using React, Vite, and Tailwind CSS to provide a responsive user interface.
- * The project aims to demonstrate the usefulness of AI in intelligent information retrieval and question answering, with a focus on reducing manual effort, improving information access, and integrating AI into document-centric workflows.

22

4.4 Results

The developed system was successfully implemented as a full-stack web application. The backend APIs correctly handled user authentication, document uploading, chat session management, and document-based response generation. The frontend provided a clean workflow for interacting with the system.

The following functional results were observed during execution:

Table 4.2 Functional Results Summary

Test Case	Expected Result	Observed Result
User registration	New user account should be created	Successful
User login	Valid user should receive access token	Successful
File upload	Supported document should be uploaded and indexed	Successful
Session creation	New chat session should be created	Successful
Ask question	Relevant answer should be generated from document	Successful
Rename session	Session title should be updated	Successful
Delete session	Session should be removed from list	Successful
Unauthorized access	Invalid token should deny access	Successful

From a usability perspective, the system simplifies document interaction. Instead of manually searching files, the user can simply ask a question in natural language. The chat-based design makes the process intuitive and suitable for non-technical users as well. From a technical perspective, the use of a vector database and embeddings improved contextual retrieval compared to plain keyword search. The inclusion of source metadata such as page numbers increases trust in the generated answers. The modular design also makes the system easier to extend in the future.

The project therefore achieves its objective of building a practical document-aware assistant using modern AI and web technologies.

CHAPTER 5

CONCLUSION AND FUTURE WORK

Conclusion

The mini project titled **MindVault: Intelligent Document Assistant Using Retrieval-Augmented Generation** was successfully designed and developed to solve the problem of extracting meaningful answers from user-uploaded documents. The system integrates authentication, document upload, text extraction, semantic indexing, retrieval, and answer generation into a single web application. It supports multiple file formats and maintains user-specific chat sessions, making it both useful and realistic as a modern document assistant.

The project demonstrates that retrieval-augmented generation is an effective technique for grounding chatbot responses in actual document content. Compared to traditional keyword-based search, the proposed system offers more natural and context-aware interaction. The use of FastAPI, React, SQLite, ChromaDB, and embedding models shows that powerful AI-assisted applications can be built with accessible and open technologies.

Overall, the project meets its aim of creating an intelligent and secure platform for document-based question answering. It also provides a strong foundation for further improvements in AI-enabled document understanding systems.

Future Work

The current system can be extended in several ways to increase its capability and practical impact.

1. OCR support can be added for scanned and image-based PDF documents.
2. Multilingual document processing can be included for regional and international language support.
3. Voice-based query input and speech output can improve accessibility.
4. Cloud deployment can be introduced for multi-user online access at larger scale.
5. Advanced summarization and report generation features can be added.
6. Role-based access control can be implemented for institutional usage.
7. Admin dashboards can be developed for monitoring uploads and usage analytics.
8. Hybrid search combining keyword and semantic retrieval can further improve accuracy.
9. Citation formatting and highlighted source preview can make answers more verifiable.
10. Model optimization can reduce latency and improve response quality.

REFERENCES

1. Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al. (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901.
2. ChromaDB Documentation. (2024). Chroma: The AI-native open-source embedding database. Retrieved from <https://docs.trychroma.com>
3. FastAPI Documentation. (2024). FastAPI framework documentation. Retrieved from <https://fastapi.tiangolo.com>
4. React Documentation. (2024). React official documentation. Retrieved from <https://react.dev>
5. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.
6. Reimers, N., and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 3982-3992.
7. SQLAlchemy Documentation. (2024). SQLAlchemy 2.0 documentation. Retrieved from <https://docs.sqlalchemy.org>
8. PyPDF Documentation. (2024). PyPDF documentation. Retrieved from <https://pypdf.readthedocs.io>
9. Pydantic Documentation. (2024). Pydantic documentation. Retrieved from <https://docs.pydantic.dev>
10. JWT Introduction. (2024). JSON Web Tokens. Retrieved from <https://jwt.io/introduction>